Create a simple API that uses a PostgreSQL connection pool using python and FastAPI.

The API must start successfully and accept HTTP requests even when PostgreSQL is unavailable.

1. Create a Docker Compose configuration containing the API and PostgreSQL. For this task, API startup must not wait for database readiness.
2. Run a single API process with one shared connection pool, limited to **5 database connections**. Reuse the pool across requests.
3. Implement the following endpoints:
   - `GET /health/live` – returns HTTP `200` while the API is running. It must not query the database.
   - `GET /health/ready` – executes `SELECT 1` through the pool and returns `200` on success or `503` when the database is unavailable.
   - `GET /db-time` – executes `SELECT NOW()` through the pool and returns the result as JSON. Return `503` when the database cannot be reached.
4. Configure timeouts so that database-dependent requests return an error within **5 seconds** during an outage.
5. Once PostgreSQL becomes available, the API must establish connections and start serving database requests without an application restart.
6. Handle a database outage after successful operation as well. Discard broken connections and recover automatically when PostgreSQL returns.
7. Close the connection pool during graceful application shutdown.

Restrictions:

- Do not create a new pool for each request.
- Do not block HTTP server startup while waiting for PostgreSQL.
- Do not use a container restart loop to achieve recovery.

Acceptance criteria: Demonstrate the following sequence without restarting the API:

Scenario	`/health/live`	`/health/ready`	`/db-time`
Start the API while PostgreSQL is stopped	`200`	`503`	`503`
Start PostgreSQL and allow connections to recover	`200`	`200`	`200` with database time
Stop PostgreSQL again	`200`	`503`	`503`
Restart PostgreSQL and allow recovery	`200`	`200`	`200` with database time

Also send **20 concurrent requests** to `/db-time` and demonstrate, using pool metrics or PostgreSQL session information, that the API's pool never exceeds 5 connections.

Questions: What is the difference between liveness and readiness? What happens when every connection in the pool is already in use?